



Security Assessment Report

Permissionless

Incremental Review

May 12, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Permissionless program changes made by PR#350 and PR#375.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 16 issues or questions.

Task	Type	Commit
Permissionless PR#350	Solana	2a28a8e (Jan 18, 2026) → ef3b0cb (Mar 3, 2026)
Permissionless PR#375	Solana	eeee1a6 (Apr 6, 2026) → afd6100 (Apr 20, 2026)

The post-audit review was conducted on commit 6df67b9 (May 12, 2026) in the release-v4.0.0-instant-withdrawal branch.

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview	3
Findings in Detail	4
[P1-L-01] Shared token program constraint prevents mode switching	4
[P1-L-02] Incorrect rent destination in remove_mode locks reclaimed lamports in PDA	5
[P1-L-03] Missing consistency check between mode_config and token_metadata	6
[P1-L-04] Missing ATA check for pool_owner_treasury causes fee fragmentation	8
[P1-I-01] Missing reconciliation checks for deployed liquidity during cleanup	9
[P1-I-02] Silent value loss during mode switch	11
[P1-I-03] Inconsistent share calculation logic between deposit and switch_mode	13
[P1-I-04] Incomplete asset refresh during deposit allows bypassing liquidity cap	15
[P1-Q-01] Strict JupLend deployment config requirement for instant withdrawals	17
[P2-M-01] State reset allows lenders to bypass lockups and instantly withdraw	19
[P2-L-01] Missing target validation allows cross-wallet manual deployments	21
[P2-I-01] Incorrect rent destination in remove_manual_strategy_manager	23
[P2-I-02] Missing ATA validation for fToken accounts	25
[P2-I-03] Missing version and canary checks in Kamino Lend offset reader	26
[P2-I-04] Misleading documentation regarding the manual deployment daily limit	28
[P2-Q-01] Clarification on protocol state checks across administrative instructions	29
Appendix: Methodology and Scope of Work	31

Result Overview

Issue	Impact	Status
PERMISSIONLESS PR#350		
[P1-L-01] Shared token program constraint prevents mode switching	Low	Resolved
[P1-L-02] Incorrect rent destination in remove_mode locks reclaimed lamports in PDA	Low	Resolved
[P1-L-03] Missing consistency check between mode_config and token_metadata	Low	Resolved
[P1-L-04] Missing ATA check for pool_owner_treasury causes fee fragmentation	Low	Resolved
[P1-I-01] Missing reconciliation checks for deployed liquidity during cleanup	Info	Acknowledged
[P1-I-02] Silent value loss during mode switch	Info	Resolved
[P1-I-03] Inconsistent share calculation logic between deposit and switch_mode	Info	Resolved
[P1-I-04] Incomplete asset refresh during deposit allows bypassing liquidity cap	Info	Acknowledged
[P1-Q-01] Strict JupLend deployment config requirement for instant withdrawals	Question	Acknowledged
PERMISSIONLESS PR#375		
[P2-M-01] State reset allows lenders to bypass lockups and instantly withdraw	Medium	Resolved
[P2-L-01] Missing target validation allows cross-wallet manual deployments	Low	Resolved
[P2-I-01] Incorrect rent destination in remove_manual_strategy_manager	Info	Resolved
[P2-I-02] Missing ATA validation for fToken accounts	Info	Resolved
[P2-I-03] Missing version and canary checks in Kamino Lend offset reader	Info	Acknowledged
[P2-I-04] Misleading documentation regarding the manual deployment daily limit	Info	Resolved
[P2-Q-01] Clarification on protocol state checks across administrative instructions	Question	Resolved

Findings in Detail

PERMISSIONLESS PR#350

[P1 -L -01] Shared token program constraint prevents mode switching

Identified in commit [0cf5e47](#).

The `switch_mode` instruction allows users to migrate their liquidity from one mode to another within the same pool. However, the current account validation struct for this instruction explicitly constrains both the source and destination mode tokens to utilize a single, shared token program.

As seen in the instruction context, both `source_mode_token` and `destination_mode_token` share the exact same `token_program` constraint:

```
/* programs/permissionless/src/pool/instructions/switch_mode.rs */
159 | #[account(
160 |     mut,
161 |     token::mint = source_mode_mint,
162 |     token::authority = lender,
163 |     token::token_program = token_program
164 | )]
165 | pub source_mode_token: Box<InterfaceAccount<'info, TokenAccount>>,
...
188 | #[account(
189 |     mut,
190 |     token::mint = destination_mode_mint,
191 |     token::authority = lender,
192 |     token::token_program = token_program
193 | )]
194 | pub destination_mode_token: Box<InterfaceAccount<'info, TokenAccount>>,
195 |
196 | pub token_program: Interface<'info, TokenInterface>,
```

Because the interface allows a pool to contain multiple modes, a pool owner could validly configure "Mode A" using the standard SPL Token program and "Mode B" using the Token-2022 program. If a user attempts to switch their liquidity between these two modes, the transaction will categorically fail during Anchor's account validation phase. This architectural flaw creates a persistent denial of service for the `switch_mode` feature whenever a pool utilizes mixed token standards.

Resolution

This issue has been resolved by [b86831d](#).

PERMISSIONLESS PR#350

[P1 -L -02] Incorrect rent destination in remove_mode locks reclaimed lamports in PDA

Identified in commit [0cf5e47](#).

The `remove_mode` instruction is designed to allow a pool owner to safely remove a mode configuration from their pool, which includes securely closing the associated `pool_mode_token` account to reclaim its storage rent. However, the current implementation incorrectly routes these reclaimed lamports to the pool's authority PDA instead of the transaction signer.

During the cleanup phase of the instruction, the protocol constructs a CPI call to the Token program to close the token account. In this `CloseAccount` struct, the `destination` field which determines where the recovered rent lamports are sent is explicitly assigned to the `pool_authority`.

```
/* programs/permissionless/src/pool/instructions/remove_mode.rs */
040 | // Close the pool mode token account.
041 | let cpi_accounts = CloseAccount {
042 |     account: ctx.accounts.pool_mode_token.to_account_info(),
043 |     destination: ctx.accounts.pool_authority.to_account_info(),
044 |     authority: ctx.accounts.pool_authority.to_account_info(),
045 | };
```

Because the `pool_authority` is an PDA used purely for program-derived authorization, it is not designed to function as a treasury or a user wallet. Consequently, any lamports transferred to this PDA during the account closure process become permanently locked within the PDA, leading to a direct financial loss for the pool owner who originally funded the account creation.

To ensure proper rent reclamation and prevent the permanent locking of funds, the `CloseAccount` configuration should be updated to use `destination: ctx.accounts.pool_owner.to_account_info()`, routing the lamports directly back to the pool owner.

Resolution

This issue has been resolved by [3dee54e](#).

PERMISSIONLESS PR#350

[P1 -L -03] Missing consistency check between mode_config and token_metadata

Identified in commit [0cf5e47](#).

The `update_mode_token_metadata` instruction enables authorized pool or protocol owners to modify the metadata attributes (name, symbol, and URI) associated with a specific mode's token. However, the instruction's account validation logic fails to enforce a structural relationship between the provided `mode_config` account and the target `token_metadata` account.

As seen in the instruction context, the `mode_config` is validated strictly by its seeds:

```
/* programs/permissionless/src/mode/instructions/update_mode_token_metadata.rs */
091 | #[account(
092 |     seeds = [
093 |         MODE_CONFIG_SEED,
094 |         pool_config.key().as_ref(),
095 |         mode_config.id.as_ref(),
096 |     ],
097 |     bump = mode_config.bump,
098 | )]
099 | pub mode_config: Box<Account<'info, ModeConfig>>,
```

Conversely, the `token_metadata` account is defined merely as an `UncheckedAccount`. The program relies exclusively on the downstream Metaplex Token Metadata CPI to validate authorization invariants:

```
/* programs/permissionless/src/mode/instructions/update_mode_token_metadata.rs */
111 | /// The token metadata account.
112 | /// CHECK: Readonly, and the PDA will be checked in the MPL token metadata program.
113 | #[account(mut)]
114 | pub token_metadata: UncheckedAccount<'info>,
```

The downstream Metaplex program will only verify that the CPI caller is the designated `update_authority` for the provided metadata account. Because a single pool can support multiple distinct modes, all mode mints within the same pool share the exact same `pool_authority` PDA as their update authority. Consequently, the Metaplex validation will succeed even if a caller submits the `mode_config` for "Mode A" but maliciously or accidentally provides the `token_metadata` account for "Mode B".

This disconnect allows for cross-mode metadata pollution within the same pool. When the mismatch occurs, the emitted `ModeTokenMetadataUpdatedEvent` will log the update against the wrong `mode_config` key, silently corrupting off-chain indexing and tracking while physically altering the wrong token's metadata on-chain.

Resolution

This issue has been resolved by [7cff624](#).

PERMISSIONLESS PR#350

[P1 -L -04] Missing ATA check for pool_owner_treasury causes fee fragmentation

Identified in commit [0cf5e47](#).

The `instant_withdraw` instruction facilitates rapid user withdrawals from a pool's mode, during which it systematically deducts and transfers a designated fee to the pool owner's treasury. However, the validation logic for the receiving `pool_owner_treasury_underlying_token` account is overly permissive, verifying only the account's mint and owner while crucially omitting an explicit check for the standard Associated Token Account (ATA) derivation.

```
/* programs/permissionless/src/mode/instructions/instant_withdraw.rs */
246 | #[account(
247 |     mut,
248 |     token::mint = underlying_mint,
249 |     token::authority = pool_config.pool_owner_treasury,
250 |     token::token_program = underlying_token_program
251 | )]
252 | pub pool_owner_treasury_underlying_token: Box<InterfaceAccount<'info, TokenAccount>>,
```

Within the Solana account model, a single public key can serve as the authority for an arbitrary number of token accounts associated with the same mint. Because the program accepts any valid token account that simply matches the required `mint` and `authority`, a malicious actor or a misconfigured frontend can permissionlessly initialize non-canonical token accounts and intentionally supply them during the withdrawal transaction. When this occurs, the protocol's collected fees are deposited into these auxiliary, fragmented accounts rather than the canonical ATA.

Resolution

This issue has been resolved by [370fb6f](#).

PERMISSIONLESS PR#350

[P1 –I –01] Missing reconciliation checks for deployed liquidity during cleanup

Identified in commit [0cf5e47](#).

The protocol allows administrators to manage external investment strategies and the pool lifecycle through the `remove_deployment_target` and `close_pool` instructions. However, both of these administrative cleanup actions fail to verify whether the outstanding deployed liquidity has been fully recalled prior to execution, creating a risk of fund loss.

In the case of `remove_deployment_target`, the instruction allows the pool or protocol owner to delete a strategy target and directly close its associated `deployment_config` and `deployment_state` accounts.

```
/* programs/permissionless/src/investment/instructions/remove_deployment_target.rs */
012 | pub(crate) fn remove_deployment_target(ctx: Context<RemoveDeploymentTarget>) -> Result<()> {
013 |     let huma_config = preconditions::only_valid_huma_config(ctx.accounts.huma_config.as_ref())?;
014 |     preconditions::only_pool_owner_or_huma_owner(
015 |         &huma_config,
016 |         ctx.accounts.pool_config.as_ref(),
017 |         &ctx.accounts.signer,
018 |     )?;
019 |
020 |     let pool_state = ctx.accounts.pool_state.as_ref();
021 |     pool_state.require_pool_not_closed()?;
022 |
023 |     let deployment_config = ctx.accounts.deployment_config.as_ref();
024 |     emit!(DeploymentTargetRemovedEvent {
025 |         pool_id: ctx.accounts.pool_config.pool_id,
026 |         strategy_type: deployment_config.strategy_type,
027 |         target_key: deployment_config.target_key,
028 |     });
029 |
030 |     Ok(())
031 | }
```

This instruction does not verify whether the strategy's `cumulative_amount_deployed` is perfectly balanced with its `cumulative_amount_paid_back`. Consequently, an administrator can inadvertently close the accounts while funds are still actively deployed in the external strategy.

A parallel vulnerability exists within the `close_pool` instruction. While it verifies that all redemption requests have been processed and that the pool vault has a sufficient physical balance to cover total internal assets, it does not explicitly enforce that all external strategy positions have been fully cleared.

```
/* programs/permissionless/src/pool/accounts.rs */
562 | pub fn close_pool(&mut self, underlying_balance: u64) -> Result<()> {
563 |     require!(self.is_pool_in_pre_closure(), Error::PoolIsNotInPreClosure);
564 |     // All redemption requests must have been processed prior to pool closure.
```

```

565 |     require!(
566 |         self.redemption.next_request_id > self.redemption.last_request_id,
567 |         Error::UnprocessedRedemptionRequests
568 |     );
569 |
570 |     // The pool must have enough underlying assets to satisfy all the withdrawal needs.
571 |     // instant_withdrawal_reserve_limit argument is set to 0 here because during pool closure, all assets
572 |     // should be considered available for withdrawal, including the instant withdrawal reserve.
573 |     let available_balance = self.get_available_balance(underlying_balance, 0);
574 |     require!(
575 |         available_balance >= self.get_pool_total_assets(),
576 |         Error::InsufficientBalanceForWithdrawal
577 |     );
578 |
579 |     self.status = PoolStatus::Closed;
580 |
581 |     Ok(())
582 | }

```

Because it omits a strict check against the deployed liquidity balance (e.g., `liquid_assets_deployed == 0`), the pool can be closed while deployed liquidity still remains out in the wild. This traps the remaining pool funds in the external strategy wallet, stranding them indefinitely.

Resolution

The team clarified that the omission is by design. When a strategy incurs a partial or total loss, deployed funds can never be fully paid back. Requiring a balanced deployment ledger before removal would make it impossible to clean up the account.

PERMISSIONLESS PR#350

[P1 -I -02] Silent value loss during mode switch

Identified in commit [0cf5e47](#).

The `switch_mode` instruction facilitates the migration of a user's liquidity from one pool mode to another by internally calculating the equivalent value, burning the source mode shares, and minting the corresponding destination mode shares. However, the instruction fails to verify that the calculated number of destination shares to mint is strictly greater than zero prior to executing the state mutations and token transfers.

During the execution flow, the instruction invokes `pool_state.switch_mode` to compute `shares_to_mint` and `assets_switched` based on the provided source shares and the relative exchange rates of the two modes. Immediately following this calculation, the protocol executes the token burns and mints:

```
/* programs/permissionless/src/pool/instructions/switch_mode.rs */
042 | let (shares_to_mint, assets_switched) = pool_state.switch_mode(
043 |     source_mode_index,
044 |     destination_mode_index,
045 |     shares,
046 |     ctx.accounts.source_mode_mint.supply,
047 |     ctx.accounts.destination_mode_mint.supply,
048 |     source_mode_config.periodic_apy_bps,
049 |     destination_mode_config.periodic_apy_bps,
050 | )?;
051 |
052 | // Burn the source mode tokens then mint destination mode tokens to the lender.
053 | token_interface::burn(
054 |     CpiContext::new_with_signer(
055 |         ctx.accounts.token_program.to_account_info(),
056 |         Burn {
057 |             mint: ctx.accounts.source_mode_mint.to_account_info(),
058 |             from: source_mode_token.to_account_info(),
059 |             authority: ctx.accounts.lender.to_account_info(),
060 |         },
061 |         &[],
062 |     ),
063 |     shares,
064 | )?;
...
072 | token_interface::mint_to(
073 |     CpiContext::new_with_signer(
074 |         ctx.accounts.token_program.to_account_info(),
075 |         MintTo {
076 |             mint: ctx.accounts.destination_mode_mint.to_account_info(),
077 |             to: ctx.accounts.destination_mode_token.to_account_info(),
078 |             authority: ctx.accounts.pool_authority.to_account_info(),
079 |         },
080 |         signer_seeds,
081 |     ),
082 |     shares_to_mint,
083 | )?;
```

When a user switches a very small number of source shares (dust), the conversion formula can round the resulting `shares_to_mint` down to exactly `0`. Because there is no explicit `require!(shares_to_mint > 0, ...)` guard in the instruction, the protocol will successfully burn the user's `shares` and invoke the `mint_to` CPI with an amount of `0`. This results in a silent and permanent loss of value for the lender, as their source liquidity is destroyed without granting them any proportional equity in the destination mode.

Resolution

This issue has been resolved by `b86831d`.

PERMISSIONLESS PR#350

[P1-I-03] Inconsistent share calculation logic between deposit and switch_mode

Identified in commit [0cf5e47](#).

The `deposit` and `switch_mode` instructions both allow users to enter a pool mode, requiring the protocol to calculate the appropriate number of shares to mint based on the incoming assets, the mode's current total assets, and the mode's current share supply. However, there is a structural inconsistency in how these two execution paths handle the specific edge case where a mode maintains a non-zero share supply but its total assets have dropped to exactly zero (e.g., following a total loss scenario).

In the standard `deposit` flow, the protocol relies on the `convert_to_shares` helper function to determine the minted amount. This function explicitly accounts for the zero-asset edge case, safely returning `0` shares, which subsequently triggers a `ZeroSharesMinted` error to protect the user:

```
/* programs/permissionless/src/pool/accounts.rs */
1033 | fn convert_to_shares(mode_assets: u128, mode_supply: u64, assets: u64) -> u64 {
1034 |     if mode_supply != 0 && mode_assets == 0 {
1035 |         return 0;
1036 |     }
1037 |     if mode_supply == 0 {
1038 |         assets
1039 |     } else {

/* programs/permissionless/src/pool/accounts.rs */
598 | let shares = Self::convert_to_shares(
599 |     self.get_mode_state_at_index(mode_index).assets,
600 |     mode_supply,
601 |     amount,
602 | );
603 | if shares == 0 {
604 |     // Disallows 0 shares to be minted. This can be caused by rounding errors or the mode
605 |     // losing all of its assets after default.
606 |     return err!(Error::ZeroSharesMinted);
607 | }
```

Conversely, the `switch_mode` logic bypasses this centralized helper and computes the destination shares inline. When evaluating the destination mode, if the assets are zero, it indiscriminately falls back to a 1:1 conversion rate (minting shares equal to `assets_switched`) regardless of whether the destination share supply is non-zero:

```
/* programs/permissionless/src/pool/accounts.rs */
720 | // `destination_total_assets` could be zero if the mode has never had any deposits.
721 | let destination_shares_to_mint = if destination_total_assets == 0 {
722 |     assets_switched
```

While the `validate_mode_supply` check ensures the destination supply is non-zero at the start of `switch_m`

ode, and this specific zero-asset state is highly unlikely to be reached under normal operating conditions, the divergence in mathematical logic introduces unnecessary technical debt and behavioral inconsistencies.

Resolution

This issue has been resolved by b86831d.

PERMISSIONLESS PR#350

[P1 –I –04] Incomplete asset refresh during deposit allows bypassing liquidity cap

Identified in commit [0cf5e47](#).

The `deposit` instruction enables lenders to supply liquidity to a specific mode within a shared pool. To enforce risk management constraints and protocol limits, the pool validates that the incoming deposit amount does not cause the pool's total accumulated assets to exceed a globally configured `liquidity_cap`. However, the current implementation fails to ensure that the global asset accounting is fully up-to-date before performing this critical validation.

During the execution of the deposit logic, the protocol updates the asset state exclusively for the target mode receiving the deposit, prior to calculating the remaining capacity:

```
/* programs/permissionless/src/pool/accounts.rs */
592 | self.refresh_assets_for_mode(mode_index, yield_bps)?;
593 |
594 | let available_cap =
595 |     liquidity_cap.saturating_sub(self.get_pool_total_assets() as u128) as u64;
596 | require!(amount <= available_cap, Error::LiquidityCapExceeded);
```

The explicit call to `self.refresh_assets_for_mode(mode_index, yield_bps)?` only accrues yield and updates the internal asset state for the specific `mode_index` being interacted with. Immediately following this targeted refresh, the protocol calculates the `available_cap` by subtracting the result of `self.get_pool_total_assets()` from the `liquidity_cap`. Because a single pool can contain multiple distinct modes, `get_pool_total_assets()` must aggregate the recorded assets across all of them. If the other, non-target modes have not been interacted with recently, their internal asset tracking will be stale and will fail to account for the yield they have accrued over time.

Consequently, the sum returned by `get_pool_total_assets()` will systematically underestimate the true, real-time value of the pool's global assets. This underestimation artificially inflates the `available_cap` variable, permitting users to execute deposits that silently breach the intended maximum liquidity capacity of the pool.

Resolution

The team clarified that the deposit cap (`liquidity_cap`) is intended as a soft economic guardrail rather than a strict security invariant, and it can naturally be exceeded via yield accrual between refresh points.

To mitigate this, the protocol relies on a cron job that refreshes all modes every two hours, bounding the staleness window. Even assuming a conservative upper bound of 20% APY, the maximum undercount is minimal (approximately 0.0046%, or roughly \$46 per \$1M). Therefore, refreshing all modes on every deposit would add unnecessary compute costs to every user transaction for a negligible precision improvement on a cap that is deliberately soft.

PERMISSIONLESS PR#350

[P1-Q-01] Strict JupLend deployment config requirement for instant withdrawals

Identified in commit [0cf5e47](#).

The `instant_withdraw` and `instant_withdraw_privileged` instructions are designed to provide lenders with immediate access to their funds, bypassing the standard asynchronous redemption queue provided there are sufficient liquid reserves available. However, the account validation structs for both instructions strictly mandate the inclusion of a `DeploymentConfig` account that is explicitly derived using the `DeploymentStrategyType::JupLend` seed.

```
/* programs/permissionless/src/mode/instructions/instant_withdraw_privileged.rs */
157 | #[account(
158 |     seeds = [
159 |         DEPLOYMENT_CONFIG_SEED,
160 |         pool_config.key().as_ref(),
161 |         &[DeploymentStrategyType::JupLend as u8],
162 |         deployment_config.target_key.as_ref(),
163 |     ],
164 |     bump = deployment_config.bump,
165 | )]
166 | pub deployment_config: Box<Account<'info, DeploymentConfig>>,

/* programs/permissionless/src/mode/instructions/instant_withdraw.rs */
185 | #[account(
186 |     seeds = [
187 |         DEPLOYMENT_CONFIG_SEED,
188 |         pool_config.key().as_ref(),
189 |         &[DeploymentStrategyType::JupLend as u8],
190 |         deployment_config.target_key.as_ref(),
191 |     ],
192 |     bump = deployment_config.bump,
193 | )]
194 | pub deployment_config: Box<Account<'info, DeploymentConfig>>,
```

Because this account is enforced as a hard requirement during Anchor's account resolution phase, these withdrawal instructions will systematically revert for any pool that has not actively registered a Jupiter Lend deployment target. This architectural constraint severely limits the flexibility of the protocol, as it completely breaks the instant withdrawal functionality for pools that either rely exclusively on other investment strategies (such as manual deployments) or simply hold sufficient idle liquidity directly within the pool vault to service the withdrawal without needing to pull from an external protocol.

We would like to clarify the intended design for this feature: Is it a strict protocol prerequisite that every pool offering instant withdrawals must maintain an active `JupLend` integration?

Resolution

The team acknowledged this limitation but clarified that it does not impact current operations, as there is currently only one pool in production, which is configured with a Jupiter Lend deployment target, and there are no plans to launch a second pool. In a future release, the protocol will add support for additional instant withdrawal liquidity sources and make the source optional, so the current approach is deemed acceptable for now.

PERMISSIONLESS PR#375

[P2-M-01] State reset allows lenders to bypass lockups and instantly withdraw

Identified in commit [afd6100](#).

Commits [0965855](#) and [72922ad](#) in the new pull request introduced a state-management flaw in the `huma` program that allows lenders to manipulate their internal accounting records, enabling them to bypass withdrawal lockup periods and circumvent the standard redemption queue. This vulnerability arises because the `close_lender_accounts` instruction does not verify the lender's current tranche token balance before authorizing the closure of their `LenderState` account.

During execution, `close_lender_accounts` solely relies on the `ready_to_close()` method, which verifies that the lender has no pending redemption requests or unwithdrawn processed funds. It notably omits a check to ensure that the lender's tranche token balance is zero.

```
/* programs/huma/src/tranche_vault/states.rs */
118 | pub fn ready_to_close(&self) -> bool {
119 |     self.redemption_record.num_shares_requested == 0
120 |     && self.redemption_record.withdrawable() == 0
121 | }
```

Because the token account is not closed and the balance is not validated, a lender can safely retain their tranche tokens while wiping their on-chain state history. The lender can then immediately call `create_lender_accounts` to initialize a completely fresh `LenderState` PDA. This fresh state implicitly zero-initializes the `deposit_record.principal` and `deposit_record.last_deposit_time` tracking fields.

This state reset opens two distinct exploit paths depending on the lender's role. For yield-distributing lenders, the `withdraw_yields` instruction calculates the withdrawable amount by subtracting the recorded principal from the total asset value of their tokens.

```
/* programs/huma/src/tranche_vault/instructions/withdraw_yields.rs */
044 | let principal_with_decimals =
045 |     ctx.accounts.lender_state.deposit_record.principal * DEFAULT_DECIMALS_FACTOR;
046 | let yields_with_decimals = assets_with_decimals.saturating_sub(principal_with_decimals);
```

Since the reset forces `deposit_record.principal` to 0, the formula `assets_with_decimals.saturating_sub(0)` effectively treats the user's entire token balance as pure yield. This allows the user to immediately withdraw their entire principal through the yield path, permanently bypassing the standard redemption queue.

Furthermore, normal users can exploit this to bypass the protocol's time-based lockup mechanisms. In

`add_redemption_request`, the program enforces a `withdrawal_lockup_period_days` restriction based on the lender's `last_deposit_time`:

```
/* programs/huma/src/tranche_vault/instructions/add_redemption_request.rs */
072 | // Checks against withdrawal lockup period.
073 | require!(
074 |     next_epoch_start_time
075 |     >= lender_state.deposit_record.last_deposit_time
076 |     + lp_config.withdrawal_lockup_period_days as u64 * SECONDS_IN_A_DAY,
077 |     Error::WithdrawTooEarly
078 | );
```

By resetting their `LenderState`, `last_deposit_time` becomes 0. The lockup validation will then evaluate against `0 + lockup_period`, trivially passing the requirement and allowing the user to initiate a redemption instantly, regardless of when their funds were actually deposited.

Resolution

This issue has been fixed by [77bf9ae](#).

PERMISSIONLESS PR#375

[P2-L-01] Missing target validation allows cross-wallet manual deployments

Identified in commit [afd6100](#).

The `deploy_liquidity_manually` instruction is designed to allow an authorized manual strategy manager to deploy pool liquidity directly into their designated wallet, bounded strictly by a `manual_deployment_per_wallet_limit` to cap the protocol's risk exposure. However, the current account validation logic fails to enforce a structural relationship between the executing manager's wallet and the destination deployment target.

As defined in the `DeployLiquidityManually` context struct, the `manual_strategy_manager` account correctly verifies that the signing `wallet` is an authorized manager.

```
/* programs/permissionless/src/investment/instructions/deploy_liquidity_manually.rs */
125 | #[account(
126 |     seeds = [
127 |         MANUAL_STRATEGY_MANAGER_SEED,
128 |         pool_config.key().as_ref(),
129 |         manual_strategy_manager.wallet.as_ref(),
130 |     ],
131 |     bump = manual_strategy_manager.bump,
132 |     has_one = wallet @ Error::StrategyManagerWalletMismatch,
133 | )]
134 | pub manual_strategy_manager: Box<Account<'info, ManualStrategyManager>>,
```

However, the `deployment_config` and its associated `deployment_state` are resolved using whichever `target_key` the caller provides in the account list:

```
/* programs/permissionless/src/investment/instructions/deploy_liquidity_manually.rs */
104 | #[account(
105 |     seeds = [
106 |         DEPLOYMENT_CONFIG_SEED,
107 |         pool_config.key().as_ref(),
108 |         &[DeploymentStrategyType::Manual as u8],
109 |         deployment_config.target_key.as_ref(),
110 |     ],
111 |     bump = deployment_config.bump,
112 | )]
113 | pub deployment_config: Box<Account<'info, DeploymentConfig>>,
```

Because the instruction never asserts that `deployment_config.target_key == wallet.key()`, a malicious or compromised manual strategy manager ("Manager A") can execute this instruction and supply the `deployment_config` belonging to a different authorized manager ("Manager B"). Since the net-amount-deployed constraint evaluates against the provided `deployment_state` (which belongs to Manager B), Manager A can successfully deploy funds into Manager B's token account.

Resolution

This issue has been fixed by [db5344f](#).

PERMISSIONLESS PR#375

[P2-I-01] Incorrect rent destination in remove_manual_strategy_manager

Identified in commit [afd6100](#).

Within the investment strategy architecture, the creation of a `ManualStrategyManager` account requires approval from the Huma protocol owner. During this approval process in the `approve_manual_strategy_manager` instruction, the storage rent for the new account is explicitly funded by the `huma_owner` via the `payer` constraint.

```
/* programs/permissionless/src/investment/instructions/approve_manual_strategy_manager.rs */
084 | #[account(
085 |     init,
086 |     seeds = [
087 |         MANUAL_STRATEGY_MANAGER_SEED,
088 |         pool_config.key().as_ref(),
089 |         proposal.wallet.as_ref(),
090 |     ],
091 |     bump,
092 |     payer = huma_owner,
093 |     space = ManualStrategyManager::DISCRIMINATOR.len() + ManualStrategyManager::INIT_SPACE,
094 | )]
095 | pub manual_strategy_manager: Box<Account<'info, ManualStrategyManager>>,
```

However, when this authorized strategy manager is eventually removed from the pool via the `remove_manual_strategy_manager` instruction, the protocol incorrectly routes the reclaimed storage lamports to the `pool_owner`.

```
/* programs/permissionless/src/investment/instructions/remove_manual_strategy_manager.rs */
054 | #[account(
055 |     mut,
056 |     seeds = [
057 |         MANUAL_STRATEGY_MANAGER_SEED,
058 |         pool_config.key().as_ref(),
059 |         manual_strategy_manager.wallet.as_ref(),
060 |     ],
061 |     bump = manual_strategy_manager.bump,
062 |     close = pool_owner,
063 | )]
064 | pub manual_strategy_manager: Box<Account<'info, ManualStrategyManager>>,
```

Because the `pool_owner` receives the lamports that were originally paid by the `huma_owner`, the removal process effectively facilitates a structural transfer of value from the protocol administrators to individual pool owners. While the financial impact of this storage rent is minimal, this accounting mismatch violates the expected symmetry of account lifecycles.

Resolution

This issue has been fixed by [2bf5de2](#).

PERMISSIONLESS PR#375

[P2-I-02] Missing ATA validation for fToken accounts

Identified in commit [afd6100](#).

Within the `JupLendStrategy` implementation, the protocol integrates with Jupiter Lend by deploying and withdrawing underlying assets in exchange for fTokens. These interactions require passing the pool authority's fToken account via the `remaining_accounts` array (`recipient_token_account` during deployment, and `owner_token_account` during withdrawal). However, the protocol does not explicitly verify that these supplied accounts are canonical ATAs.

The underlying Jupiter Lend program validates that the provided account is an token account owned by the calling authority, but it does not strictly enforce that it must be the standardized ATA for that mint. Because of this permissive upstream validation, a caller or frontend could potentially supply a non-canonical token account owned by the `pool_authority` PDA.

While this does not introduce a direct risk of fund theft, since the PDA still retains ownership of the fTokens, it creates a vector for token fragmentation. If fTokens are deposited into multiple non-standard accounts over time, the protocol's accounting may become convoluted, and executing a full withdrawal would require discovering and consolidating these fragmented balances.

Resolution

This issue has been fixed by [1c2a4d5](#).

PERMISSIONLESS PR#375

[P2-I-03] Missing version and canary checks in Kamino Lend offset reader

Identified in commit [afd6100](#).

The `KaminoLendStrategy` relies on manual byte-offset reading to parse the `Reserve` account data due to dependency conflicts. While the currently implemented offsets precisely target the `Reserve` for KLen, the protocol lacks defensive sanity checks to guarantee that the un-deserialized data blob still adheres to this specific layout.

The `read_reserve_liquidity_fields` function directly slices the byte array at hardcoded offsets to extract the values required for the fractional exchange rate calculation:

```
/* programs/permissionless/src/investment/strategies/kamino_lend_strategy.rs */
260 | fn read_reserve_liquidity_fields(
261 |     reserve_info: &AccountInfo,
262 | ) -> Result<(u64, u128, u128, u128, u128, u64)> {
263 |     let data = reserve_info.data.borrow();
264 |     let read_u64 = |off: usize| u64::from_le_bytes(data[off..off + 8].try_into().unwrap());
265 |     let read_u128 = |off: usize| u128::from_le_bytes(data[off..off + 16].try_into().unwrap());
266 |
267 |     Ok((
268 |         read_u64(TOTAL_AVAILABLE_AMOUNT_OFFSET),
269 |         read_u128(BORROWED_AMOUNT_SF_OFFSET),
270 |         read_u128(ACCUMULATED_PROTOCOL_FEES_SF_OFFSET),
271 |         read_u128(ACCUMULATED_REFERRER_FEES_SF_OFFSET),
272 |         read_u128(PENDING_REFERRER_FEES_SF_OFFSET),
273 |         read_u64(MINT_TOTAL_SUPPLY_OFFSET),
274 |     ))
275 | }
```

If the Kamino protocol undergoes a major upgrade to a `v2` program that rearranges the `Reserve` struct or inserts new fields, the `Permissionless` program's hardcoded offsets will immediately point to incorrect segments of memory. Because KLen's internal `refresh_reserves_batch` checks `reserve.version == PROGRAM_VERSION`, an upstream upgrade would pass Kamino's internal checks but cause the `Permissionless` program to read garbage data.

To construct a resilient integration, it's recommended to implement canary checks within `read_reserve_liquidity_fields`. Specifically, it should read the `version` field and assert it matches the expected version (`1`), and additionally read the collateral mint pubkey at the known offset and assert it matches the provided `reserve_collateral_mint_info.key()` to confirm the byte map is perfectly aligned.

Resolution

The team acknowledged this finding, and noted that the Kamino team confirmed they avoid breaking changes to account layout. They plan to integrate the `klend-interface` crate in the next release to eliminate raw byte offset reads.

PERMISSIONLESS PR#375

[P2 –I –04] Misleading documentation regarding the manual deployment daily limit

Identified in commit [afd6100](#).

Within the `permissionless` program, the `set_manual_deployment_limits` instruction is used to establish safety constraints on how much liquidity a manual strategy manager can deploy. The documentation for this instruction explicitly describes the `daily_limit` parameter as operating on a "rolling 24-hour window".

However, analyzing the underlying implementation in the `gate_manual_deployment` method reveals that the protocol does not utilize a true rolling window. Instead, it relies on a discrete, calendar-based daily reset bounded by UTC midnight.

```
/* programs/permissionless/src/pool/accounts.rs */
838 | pub fn gate_manual_deployment(&mut self, amount: u64, daily_limit: u64) -> Result<()> {
839 |     let start_of_today = common_utils::get_start_of_today()?;
840 |     if start_of_today > self.manual_daily_window_starts_at {
841 |         self.manual_daily_amount_deployed = 0;
842 |         self.manual_daily_window_starts_at = start_of_today;
843 |     }
844 |
845 |     let new_daily_total = self.manual_daily_amount_deployed.saturating_add(amount);
...
```

While this discrete calendar-day reset is a common, efficient design choice in Solana programs and does not constitute a security vulnerability, the discrepancy between the code and its documentation can mislead integrators, auditors, or front-end developers regarding the protocol's risk exposure timing.

Resolution

This issue has been fixed by [f88c188](#).

PERMISSIONLESS PR#375

[P2-Q-01] Clarification on protocol state checks across administrative instructions

Identified in commit [afd6100](#).

The `permissionless` program implements several administrative instructions to manage the pool life-cycle, investment strategies, and configuration parameters. Within these administrative functions, the protocol relies on two primary state validations: ensuring the pool itself is not closed (`require_pool_not_closed()`), and ensuring the global Huma protocol is not paused (`huma_config.require_protocol_on()`).

While `require_pool_not_closed()` is applied broadly across most administrative actions, the application of the global `require_protocol_on()` check appears inconsistent.

For example, the following instructions strictly enforce that the global Huma protocol must be active:

- `propose_manual_strategy_manager`
- `approve_manual_strategy_manager`
- `remove_manual_strategy_manager`
- `cancel_manual_strategy_manager_proposal`
- `add_strategy_manager_wallet`
- `remove_strategy_manager_wallet`

Conversely, the following closely related administrative instructions interact with the Huma configuration but do *not* require the protocol to be active:

- `add_deployment_target`
- `remove_deployment_target`
- `add_async_deployment_target`
- `remove_async_deployment_target`

We would like to understand the design rationale governing when the global `huma_config` state must be checked, and when it can be safely bypassed. Specifically, is it necessary to lock actions like `remove_manual_strategy_manager` or `cancel_manual_strategy_manager_proposal` when the global protocol is paused?

Resolution

This issue has been fixed by [6589723](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderrect Inc. d/b/a Sec3 (the "Company") and Huma Technologies LLC (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

